

ADITYA UNIVERSITY

Department of Artificial Intelligence & Machine Learning

Smart E-Commerce Order Management System

Database Management Systems — Capstone Project

COURSE

DBMS (250IIT05)

REVIEW

Review – I : Database Design



TEAM MEMBERS

A VENKATA NAVEEN
N.KRANTHI
N.PHANINDRA
GSN KISHORE

ROLL NO

25B11AI075
25B11AI606
25B11AI839
25B11AIA79

Problem Statement

E-commerce platforms handle large volumes of information related to customers, products, orders, payments, and deliveries. Managing these operations efficiently requires a structured database that maintains relationships between entities while ensuring data consistency and integrity.

The proposed **Smart E-Commerce Order Management System** provides a centralized relational database for managing customers, products, categories, orders, payments, addresses, sellers, and shipments — simplifying order management with accurate relationships across every stage.

The system must maintain accurate relationships between customers and orders, products and order items, payments, and shipment information.

1

Scattered customer and order information

2

Difficulty tracking order status

3

Product and stock management challenges

4

Payment and shipment information management

5

Data redundancy and inconsistency

6

Need for reliable relational database management

Project Objectives

1

Maintain customer information efficiently

2

Ensure data integrity using Primary Keys, Foreign Keys and constraints

3

Maintain relationships between entities

4

Maintain individual items associated with each order

5

Record and manage payment information

6

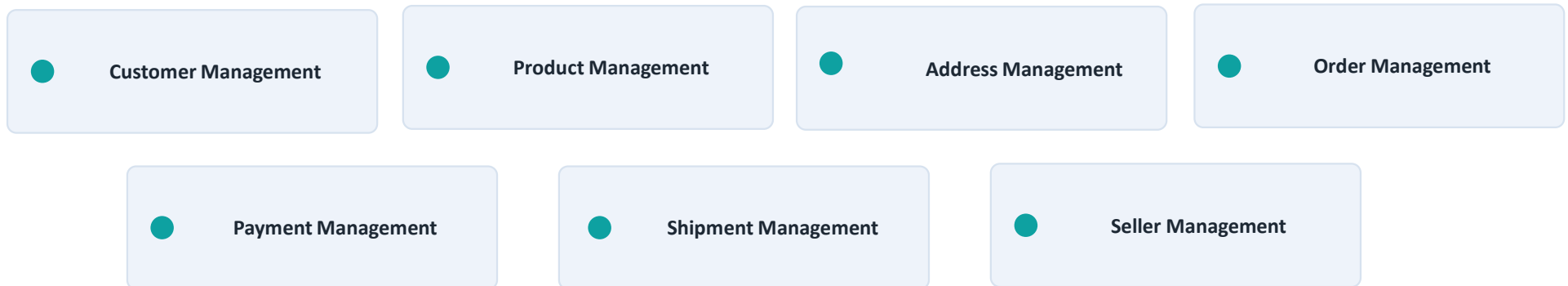
Manage customer delivery addresses

Major E-Commerce operations

The database acts as the central layer connecting all major e-commerce operations.

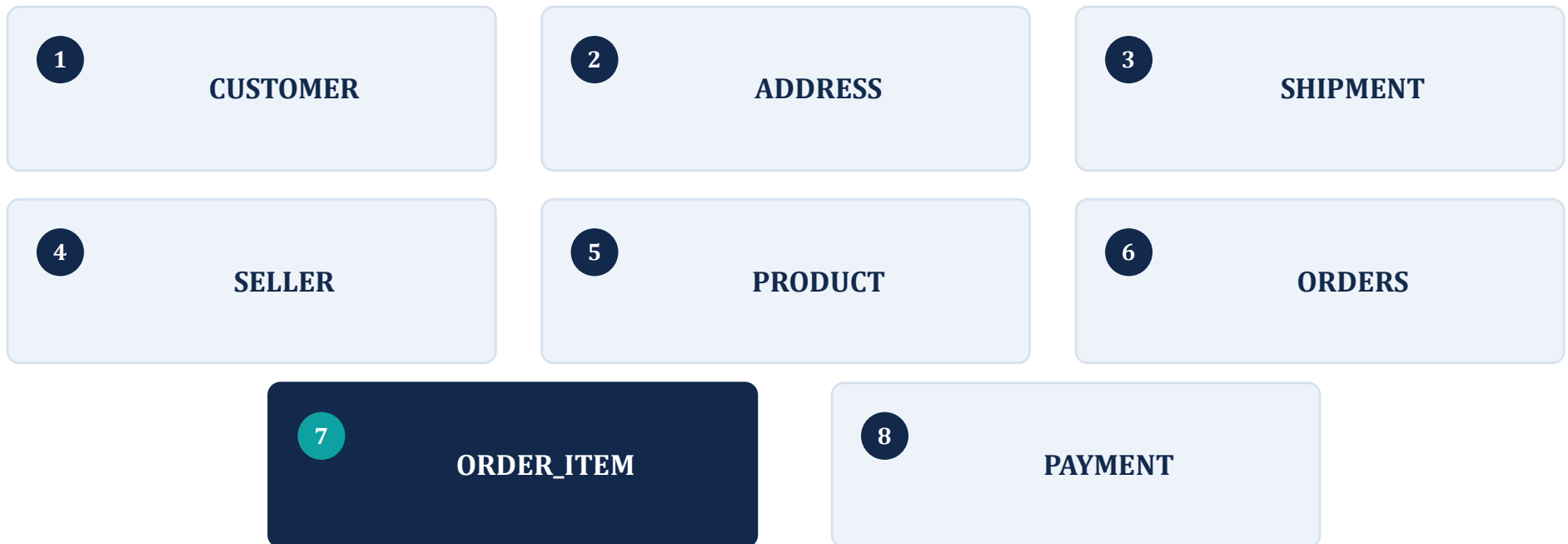


MAJOR DATABASE MODULES



Entities Identified

We have identified 8 major entities that represent the core data objects required to manage the e-commerce ordering process.



Entities and Their Attributes (1 / 2)

PK = Primary Key FK = Foreign Key

Customer

- **PK** **Customer_ID**
- First_Name
- Last_Name
- Email
- Phone
- Password
- Registration_Date

Address

- **PK** **Address_ID**
- **FK** **Customer_ID**
- House_No
- Street
- City
- State
- Pincode
- Address_Type

Category

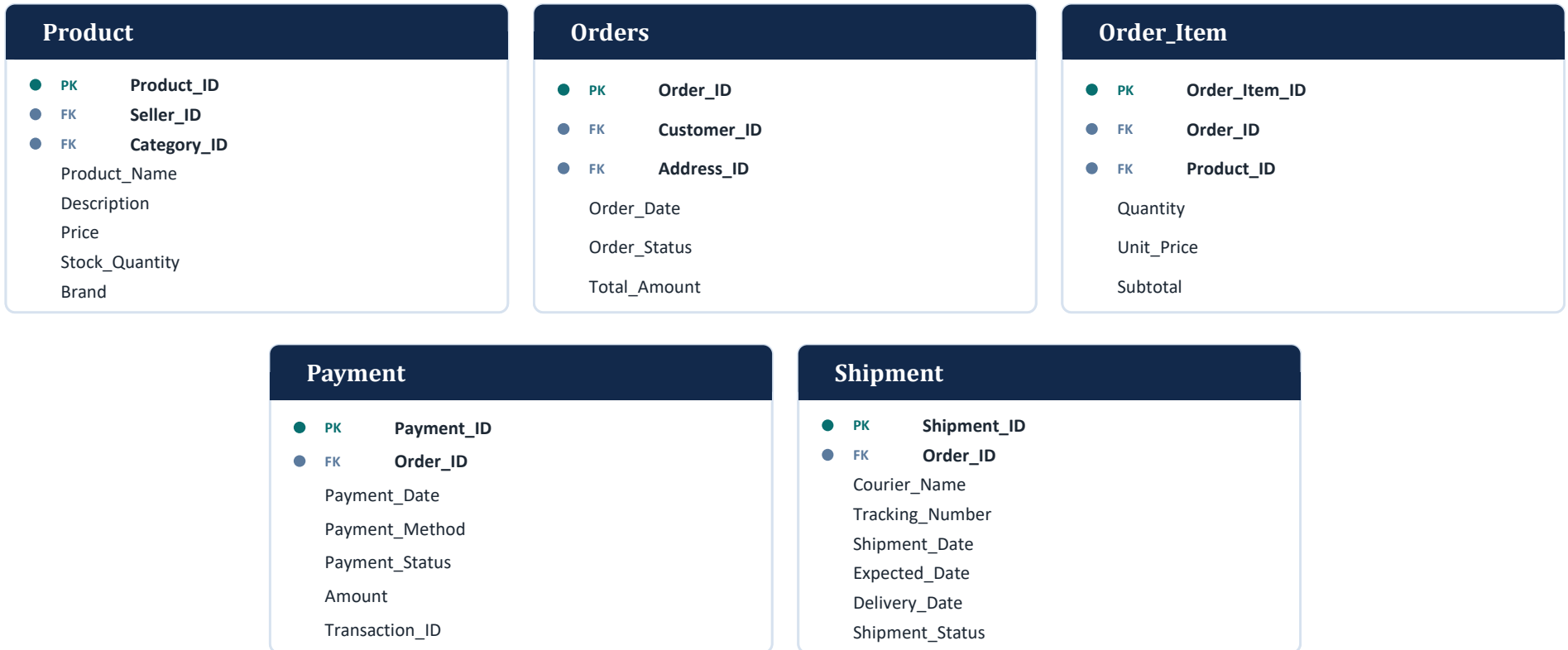
- **PK** **Category_ID**
- Category_Name
- Description

Seller

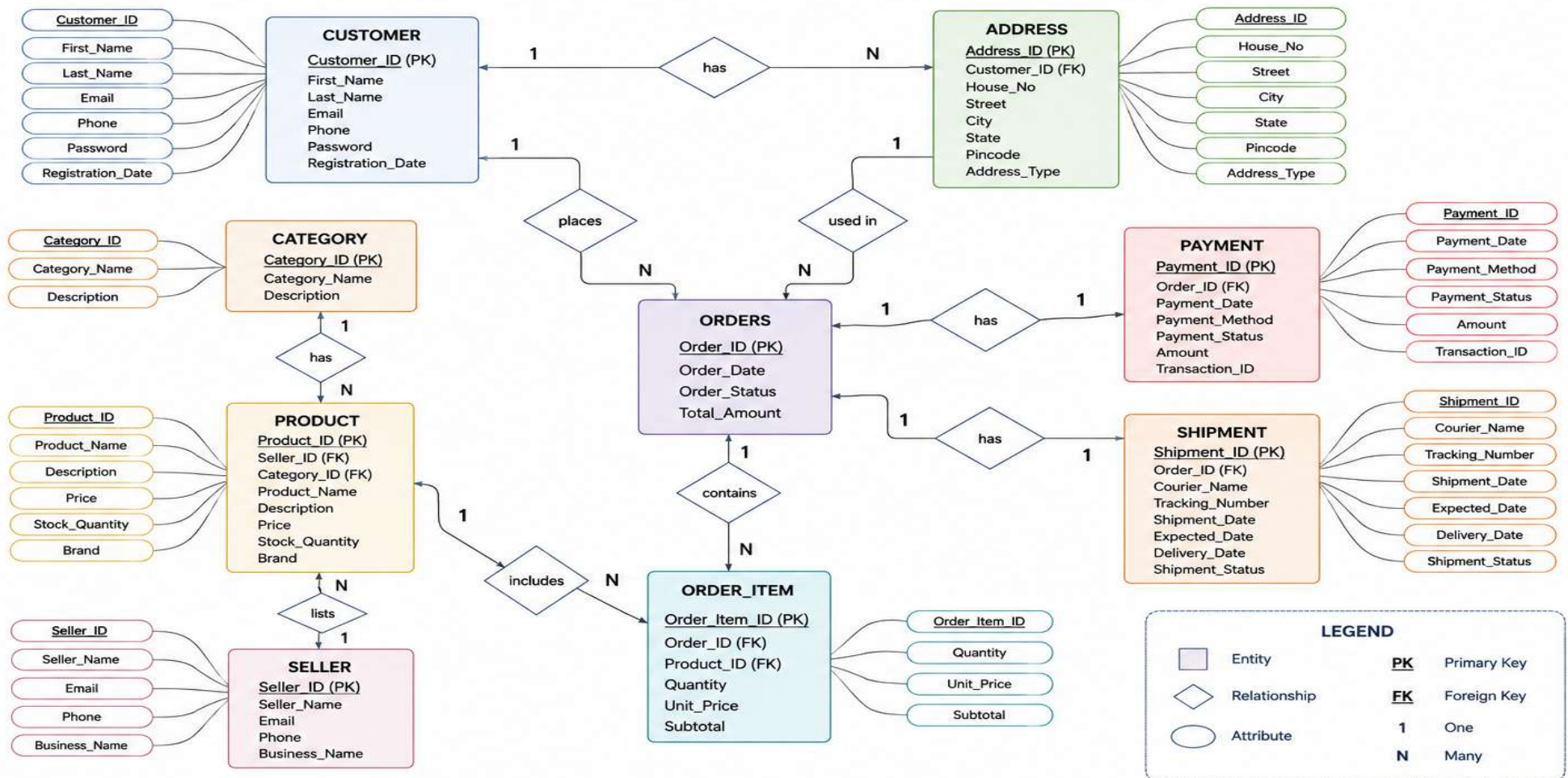
- **PK** **Seller_ID**
- Seller_Name
- Email
- Phone
- Business_Name

Entities and Their Attributes (2 / 2)

PK = Primary Key FK = Foreign Key



Entity Relationship Diagram



Relational Database Schema

CUSTOMER

PK Customer_ID
First_Name
Last_Name
Email
Phone
Password
Registration_Date

ADDRESS

PK Address_ID
FK Customer_ID
House_No
Street
City
State
Pincode
Address_Type

CATEGORY

PK Category_ID
Category_Name
Description

SELLER

PK Seller_ID
Seller_Name
Email
Phone
Business_Name

PRODUCT

PK Product_ID
FK Seller_ID
FK Category_ID
Product_Name
Description
Price
Stock_Quantity
Brand

ORDERS

PK Order_ID
FK Customer_ID
FK Address_ID
Order_Date
Order_Status
Total_Amount

ORDER_ITEM

PK Order_Item_ID
FK Order_ID
FK Product_ID
Quantity
Unit_Price
Subtotal

PAYMENT

PK Payment_ID
FK Order_ID
Payment_Date
Payment_Method
Payment_Status
Amount
Transaction_ID

SHIPMENT

PK Shipment_ID
FK Order_ID
Courier_Name
Tracking_Number
Shipment_Date
Expected_Date
Delivery_Date
Shipment_Status

Primary Keys & Foreign Keys

TABLE	PRIMARY KEY	FOREIGN KEYS
CUSTOMER	Customer_ID	—
ADDRESS	Address_ID	Customer_ID → CUSTOMER
CATEGORY	Category_ID	—
SELLER	Seller_ID	—
PRODUCT	Product_ID	Seller_ID → SELLER Category_ID → CATEGORY
ORDERS	Order_ID	Customer_ID → CUSTOMER Address_ID → ADDRESS
ORDER_ITEM	Order_Item_ID	Order_ID → ORDERS Product_ID → PRODUCT
PAYMENT	Payment_ID	Order_ID → ORDERS
SHIPMENT	Shipment_ID	Order_ID → ORDERS



Primary Key

Uniquely identifies each record in a table.
No two rows can share the same value.



Foreign Key

Creates a relationship between two tables and maintains referential integrity across the database.

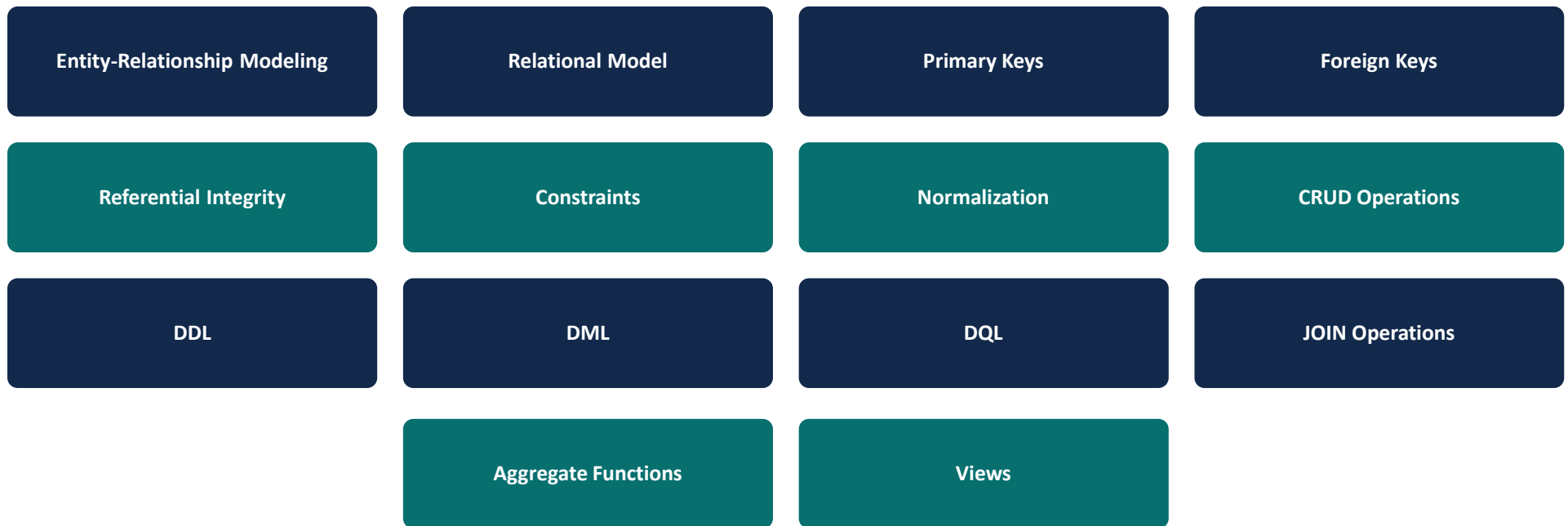
Relationships and Cardinality

Customer	1 : N	Address	One customer can have multiple addresses.
Customer	1 : N	Orders	One customer can place multiple orders.
Category	1 : N	Product	One category can contain multiple products.
Seller	1 : N	Product	One seller can list multiple products.
Orders	1 : N	Order_Item	One order can contain multiple order items.
Product	1 : N	Order_Item	One product can appear in multiple order items.
Orders	1 : 1	Payment	Each order has exactly one payment record.
Orders	1 : 1	Shipment	Each order has exactly one shipment record.

Key Concept: Orders ↔ Product is a Many-to-Many (M : N) relationship — resolved using the **ORDER_ITEM** bridge entity.

DBMS Concepts Applied

The proposed design is structured to reduce redundancy and maintain consistency through proper relationships and keys.



Team Responsibilities

1

Member 1

[N.PHANINDRA]
[25B11AI839]

Database Design

- ER Diagram
- Entities and Attributes
- Relationships
- Cardinality

2

Member 2

[G.S.N.KISHORE]
[25B11AI79]

Database Schema

- Relational Schema
- Primary Keys
- Foreign Keys
- Constraints

3

Member 3

[N.KRANTHI]
[25B11AI606]

SQL Implementation

- DDL
- DML
- DQL
- CRUD, JOINS & Aggregate Queries

4

Member 4

[A.NAVEEN]
[25B11AI075]

Documentation & Presentation

- Project Documentation
- GitHub Repository
- PPT & Screenshots
- Final Presentation

GitHub Repository

ecommerce-order-management-system/

- README.md
- **documentation/**
 - problem-statement.md
 - objectives.md
 - er-diagram.png
 - relational-schema.png
- **sql/**
 - create_tables.sql
 - insert_data.sql
 - queries.sql
 - views.sql
- **presentation/**
 - review-1.pptx

REPOSITORY PURPOSE

Version control for the project — documentation, SQL scripts, and presentation files are organized and tracked collaboratively across the team.

GITHUB REPOSITORY LINK

<https://github.com/E-commerce-Order-management-system/Smart-E-Commerce-Order-Management-System.git>

Team responsibilities are tracked through commit history, issues, and pull requests on the repository.

Conclusion

The **Smart E-Commerce Order Management System** provides a structured relational database for managing customers, products, orders, payments, addresses, sellers and shipments.

The proposed database design establishes clear relationships between entities using Primary Keys and Foreign Keys, while supporting data integrity and future SQL-based operations.

This design provides a strong foundation for the implementation phase in Review-II.

9 Entities

ER Diagram

Relational Schema

PK & FK Defined

GitHub Repo

Thank You

Questions & Discussion